



Egypt-Japan University of Science and Technology (E-JUST)

Mechatronics Lab

LAB 1

Introduction to MATLAB & Simulink

Lab Report

Name: Mohamed Nasser Mohamed

ID: 120240290

Section: 3

Task 1

Problem Statement

Given $A = \begin{bmatrix} 2 & 1 & -1; & -3 & -1 & 2; & -2 & 1 & 2 \end{bmatrix}$ and $b = \begin{bmatrix} 8; & -11; & -3 \end{bmatrix}$, solve $Ax = b$ and verify your answer by computing $A*x - b$.

Procedures

1. Defining the matrix A and the column vector b in the MATLAB workspace.
2. Using the left division operator ($\backslash$) to solve the linear system of equations for x .
3. Multiplying matrix A by x and subtract b to verify that the residual error is approximately zero.

Expected Output

- Result of x as a 3×1 vector.
- The verification calculation $A*x - b$ should output a 3×1 vector where all elements are zero (or extremely close to zero due to floating-point precision).

MATLAB Code

```
1 %% Task 1
2 clear; clc;
3
4 A = [2 1 -1; -3 -1 2; -2 1 2];
5 b = [8; -11; -3];
6
7 x = A \ b;
8
9 verf = A*x - b;
10
11 disp('x = ')
12 disp(x)
13
14 disp("Verification (A*x -b) = ")
15 disp(verf)
16
```

Figure 1: MATLAB script for Task 1

Results

```
Command Window
x =
    2.0000
    3.0000
   -1.0000

Verification (A*x -b) =
    1.0e-15 *
         0
         0
    0.8882
```

Figure 2: Command Window output for Task 1

Task 2

Problem Statement

Plot $y = e^{-0.3t} \cdot \cos(2t)$ together with its envelope $\pm e^{-0.3t}$ on one figure for $t = 0$ to 15 s. Use different line styles and a legend.

Procedures

1. Generating a time vector t from 0 to 15 seconds with 0.1 step for smooth signal.
2. Calculating the decaying cosine signal y and its upper and lower envelopes using element-by-element operators ($.*$).
3. Plotting all three signals on the same figure using the `hold on` command.
4. Applying different line styles, axis labels, a title, and a legend to distinguish the plot lines.

Expected Output

- A single plot displaying an exponentially decaying cosine wave perfectly bounded by two exponentially decaying curves (the envelopes).
- Proper formatting of the figure including axes labels, a title, a legend, and distinct line styles for each curve.

MATLAB Code

```
17 %% Task 2
18
19 t = 0:0.1:15;
20 y = exp(-0.3*t).*cos(2*t);
21 positive_env = +exp(-0.3*t);
22 negative_env = -exp(-0.3*t);
23
24
25 figure;
26 plot(t, y, 'b-', 'LineWidth', 1.5);
27 hold on;
28 plot(t, positive_env, 'r--', 'LineWidth', 1.5);
29 plot(t, negative_env, 'g-', 'LineWidth', 1.5);
30 hold off;
31
32 title('Task 2 Plot')
33 xlabel('Time (s)');
34 ylabel('Amplitude');
35 legend('y = exp(-0.3t).cos(2t)', '+exp(-0.3*t)', '-exp(-0.3*t)');
36 grid on;
37
```

Figure 3: MATLAB script for Task 2

Results

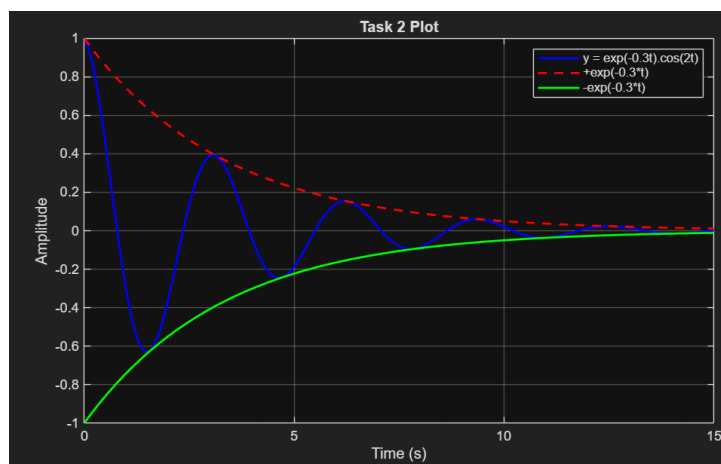


Figure 4: Generated plot figure for Task 2

Task 3

Problem Statement

Write a for loop that computes the sum of the squares of the numbers 1 to 100. Verify your result with `sum((1:100).^2)`.

Procedures

1. Initializing a sum variable to 0 in the workspace.
2. Creating a for loop with an index iterating from 1 to 100 to square each index and accumulate it into the sum variable.
3. Calculating the verification result using the built-in expression `sum((1:100).^2)`.
4. Displaying both computed results to confirm they match.

Expected Output

- The calculated numerical sum derived from the for loop.
- The calculated numerical sum derived from the vectorized verification method, which should be identical to the loop output (338350).

MATLAB Code

```
38 %% Task 3
39
40 loop_sum = 0;
41
42 for i = 1:100
43     loop_sum = loop_sum + i^2;
44 end
45
46 verification = sum((1:100).^2);
47
48 fprintf('Sum of squares using for loop: %d\n', loop_sum);
49 fprintf('Sum of squares using sum((1:100).^2): %d\n', verification);
50
```

Figure 5: MATLAB script for Task 3

Results

```
Command Window
Sum of squares using for loop: 338350
Sum of squares using sum((1:100).^2): 338350
```

Figure 6: Command Window output for Task 3